

# Buffer Overflow — From Zero to Hero Guide

## A Complete Beginner-Friendly Walkthrough

### PART 1: Computer Fundamentals (Before We Start)

#### 1.1 What Is a Computer Program?

A computer program is simply a list of instructions that the CPU (Central Processing Unit) executes one by one. When you write code in C, Python, or any language, it eventually becomes **machine code** — binary numbers that the CPU understands.

```
Source Code (C) → Compiler (gcc) → Machine Code (binary) → CPU executes
```

Every program needs **memory** to work. Think of memory (RAM) as a giant notebook with numbered pages. Each page has an **address** (its page number) and can store a small piece of data.

#### 1.2 What Is Memory?

RAM (Random Access Memory) is where your program lives while it runs. It is divided into **bytes** — each byte is 8 bits, and each byte has a unique address.

|          |        |        |        |        |        |     |
|----------|--------|--------|--------|--------|--------|-----|
| Address: | 0x0000 | 0x0001 | 0x0002 | 0x0003 | 0x0004 | ... |
| Content: | [ 48 ] | [ 65 ] | [ 6C ] | [ 6C ] | [ 6F ] | ... |
|          | H      | e      | l      | l      | o      |     |

When you create a variable like `int x = 42;`, the computer reserves a spot in memory and stores 42 there.

#### 1.3 How Is Memory Organized for a Program?

When a program runs, the operating system divides memory into sections:

|                                                   |                                   |
|---------------------------------------------------|-----------------------------------|
| +-----+ Highest Address (e.g., 0xFFFFFFFFFFFFFFF) |                                   |
| STACK                                             | ← Local variables, function calls |
| ↓ grows down                                      |                                   |
| (free space)                                      |                                   |
| ↑ grows up                                        |                                   |
| HEAP                                              | ← Dynamic memory (malloc, new)    |
| +-----+                                           |                                   |
| BSS                                               | ← Uninitialized global variables  |
| +-----+                                           |                                   |
| DATA                                              | ← Initialized global variables    |
| +-----+                                           |                                   |
| TEXT (CODE)                                       | ← Your program's instructions     |
| +-----+ Lowest Address (e.g., 0x0000000000000000) |                                   |

**The Stack** is where the action happens for buffer overflow. Let us understand it deeply.

## 1.4 What Is the Stack?

The **stack** is like a stack of plates in a cafeteria:

- You can only add a plate on **top** (PUSH)
- You can only remove from the **top** (POP)
- This is called **LIFO** — Last In, First Out

Every time your program **calls a function**, the CPU pushes a new **stack frame** onto the stack. This frame contains:

1. The function's local variables (including buffers)
2. The **saved base pointer** (RBP) — so we can return to the caller's frame
3. The **return address** — the most critical piece: WHERE to continue executing after this function finishes

```
main() calls greet() calls strlen()
```

Stack grows downward:

|                  |                                      |
|------------------|--------------------------------------|
| +-----+          |                                      |
| main()'s frame   | ← bottom of stack (placed first)     |
| +-----+          |                                      |
| greet()'s frame  | ← placed on top of main              |
| +-----+          |                                      |
| strlen()'s frame | ← placed on top of greet (current)   |
| +-----+          | ← TOP of stack (Stack Pointer - RSP) |

When `strlen()` finishes, its frame is removed and execution continues in `greet()` at the **return address** that was saved.

## 1.5 What Is a Buffer?

A **buffer** is just a block of memory reserved to hold data. In C:

```
char name[64];    // A buffer: 64 bytes reserved for a name
```

The word "buffer" comes from the concept of a temporary holding area. Data goes into the buffer temporarily before being processed.

## 1.6 What Are Registers?

The CPU has small, super-fast storage locations called **registers**. On 64-bit systems, the important ones for us:

| Register        | Full Name           | Purpose                                       |
|-----------------|---------------------|-----------------------------------------------|
| <b>RIP</b>      | Instruction Pointer | Points to the NEXT instruction to execute     |
| <b>RSP</b>      | Stack Pointer       | Points to the TOP of the stack                |
| <b>RBP</b>      | Base Pointer        | Points to the BASE of the current stack frame |
| <b>RAX</b>      | Accumulator         | General purpose + return values               |
| <b>RDI, RSI</b> | -                   | Used to pass arguments to functions           |

**RIP is the most critical register.** Whoever controls RIP controls what the program does next. This is the attacker's ultimate target.

## 1.7 What Happens When a Function Is Called?

Let us trace what happens when `main()` calls `vulnerable_function("hello")`:

```
Step 1: PUSH arguments onto stack / registers → "hello" passed in RDI
Step 2: CALL instruction                      → Pushes return address onto stack
Step 3: PUSH RBP                             → Save main()'s frame pointer
Step 4: Set RBP = RSP                        → New frame starts here
Step 5: SUB RSP, 128                         → Make room for local buffer[128]
Step 6: Execute function body                → The function does its work
Step 7: Set RSP = RBP                        → Discard local variables
Step 8: POP RBP                             → Restore main()'s frame pointer
Step 9: RET                                  → Pop return address into RIP → continue in main()
```

**The RET instruction** pops the top of the stack into RIP. If an attacker has overwritten that value, the CPU jumps to the attacker's chosen address.

## PART 2: What Is Buffer Overflow?

### 2.1 The Core Concept

Now that we understand the stack, buffer overflow is simple:

**If you write more data into a buffer than it can hold, the extra data overwrites whatever is next to it in memory.**

Since the stack places the buffer **BELOW** the return address, overflowing the buffer overwrites the return address.

| BEFORE overflow: | AFTER overflow (144 bytes into 128-byte buffer): |
|------------------|--------------------------------------------------|
| +-----+          | +-----+                                          |
| Return Address   | 0x41414141... *   ← OVERWRITTEN with "AAAA"!     |
| = 0x00401060     |                                                  |
| +-----+          | +-----+                                          |
| Saved RBP        | 0x41414141...   ← OVERWRITTEN                    |
| = 0x7FFFFFFF     |                                                  |
| +-----+          | +-----+                                          |
| buffer[128]      | AAAAAAAAAA   ← FILLED                            |
| = empty          | AAAAAAAAAA                                       |
|                  | AAAAAAAAAA                                       |
| +-----+          | +-----+                                          |

When the function tries to return, it pops `0x4141414141414141` into RIP. The CPU tries to execute code at address `0x4141414141414141`. On 64-bit systems, this is an invalid (non-canonical) address, which causes an immediate crash (Segmentation Fault) exactly on the `ret` instruction.

But what if instead of "AAAA", we put a REAL address — the address of OUR code? Then the CPU would execute OUR instructions. That is the entire attack.

### 2.2 Why Does C Allow This?

C was designed in the 1970s for maximum speed. Functions like `strcpy()` copy bytes until they hit a null byte (`\0`) — they do NOT check if the destination buffer is big enough. This design choice prioritized performance over safety.

```
// DANGEROUS: strcpy does not check buffer size
char buffer[128];
strcpy(buffer, user_input); // If user_input > 128 bytes → OVERFLOW

// SAFE: strncpy limits the copy to buffer size
char buffer[128];
strncpy(buffer, user_input, sizeof(buffer) - 1);
buffer[sizeof(buffer) - 1] = '\0';
```

Modern languages like Python, Java, and Rust prevent this by checking array bounds automatically. C and C++ do not.

---

## PART 3: The 64-bit Docker Pentest Lab

---

To understand buffer overflow safely, we provide a containerized lab environment.

### 3.1 The Lab Architecture

Our lab consists of an attacker machine and four target servers representing progressive security states:

- **Attacker Container:** Fully loaded with exploitation tools (pwntools, GDB, pwndbg, checksec).
- **Victim Level 1:** No protections. Vulnerable to classic Shellcode Injection.
- **Victim Level 2:** NX Enabled. The stack is not executable. Requires ROP (Return-Oriented Programming).
- **Victim Level 3:** NX + Canary + PIE. Requires an Information Leak followed by ROP.
- **Safe Server:** Fully mitigated. Defends against all attacks.

### 3.2 Starting the Lab

```
cd college-pentest-lab
docker compose up --build -d

# Enter the attacker container:
docker exec -it attacker bash
```

Inside the attacker container, the `/workspace/scripts/` directory contains all the exploits.

---

## PART 4: The Attack Step by Step

---

### 4.1 Reconnaissance and Fuzzing

First, we need to find out if the server crashes. We send increasingly large amounts of data to the `TRUN` command.

```
# Inside attacker container
python3 scripts/01_fuzzer.py victim-l1 9999
```

\*Output: Server crashes between 100 and 150 bytes.\*

## 4.2 Finding the Exact Offset

We know it crashes, but we need the EXACT position of the return address. We use a **cyclic pattern** (de Bruijn sequence) — a string where every 4-byte window is unique.

```
python3 scripts/02_find_offset.py victim-l1 9999
```

When the server crashes with this pattern, we look at the stack pointer (RSP) at the exact moment of the crash. If RSP points to `0x6161616261616161`, we can ask our tools where that specific string is in our pattern. In our lab, the offset is **136 bytes** (128 bytes for the buffer + 8 bytes for the saved RBP).

## 4.3 Level 1 Exploit: Shellcode Injection

**What is Shellcode?** Tiny machine code that does something useful for the attacker — usually opening a shell (command prompt).

In Level 1, there are no protections. We can simply:

1. Place our shellcode directly onto the stack.
2. Overwrite the Return Address (RIP) to point to the address of our shellcode on the stack.

```
python3 scripts/03_exploit_level1.py victim-l1 9999
```

\*Result: We get a shell on the victim server!\*

---

# PART 5: Advanced Mitigations and Bypasses

Modern systems employ **Defense in Depth** — multiple layers of security. Here is how they work, and how advanced attackers bypass them.

## 5.1 Mitigation 1: DEP/NX (Data Execution Prevention)

**How it works:** Memory pages have permissions — Read (R), Write (W), and Execute (X). NX enforces that the stack is writable but NOT executable. If we try to run our shellcode from Level 1, the CPU refuses to execute it and crashes.

### The Bypass (Level 2): ROP Chains

Since we cannot execute our own code, we execute the program's *\*existing\** code. We find tiny snippets of executable code that end in a `ret` instruction (called **gadgets**). By placing the addresses of these gadgets on the stack, we chain them together to call functions like `system("/bin/sh")` that already exist in the C standard library.

```
# Try the ROP exploit
python3 scripts/04_exploit_level12.py victim-12 9999
```

## 5.2 Mitigation 2: Stack Canaries

**How it works:** The compiler inserts a random value (the "canary") between the buffer and the return address. Before the function returns, it checks if the canary value changed. If we try to overflow the buffer, we unavoidably overwrite the canary, and the program aborts.

### The Bypass (Level 3): Information Leaks

If we can exploit a different vulnerability (like a Format String bug in the `INFO %p.%p` command) to read the stack's memory \*before\* we send our overflow, we can discover the canary's random value. We then include that exact value in our overflow payload, tricking the server into thinking nothing was corrupted.

## 5.3 Mitigation 3: ASLR and PIE (Randomization)

**How it works:** The OS randomizes the base addresses of the stack, libraries (ASLR), and the program itself (PIE) every time it runs. We cannot hardcode the addresses of our ROP gadgets because they change on every execution.

### The Bypass (Level 3): Dynamic Calculation

Our format string leak from the previous step can also leak memory addresses. If we leak an address that we know is exactly 1,000 bytes away from the start of the binary, we can subtract 1,000 to find the true, randomized base address. From there, we construct our ROP chain on the fly.

```
# Exploit Canary + PIE + NX in one shot!
python3 scripts/05_exploit_level13.py victim-13 9999
```

## 5.4 The Ultimate Defense: The Safe Server

The `safe_server.c` combines compiler mitigations (Canary, NX, PIE, RELRO) with **secure coding practices**:

1. It uses `strncpy` with proper length limits.
2. It validates input sizes before copying.
3. It fixes the format string by using `%s` safely.

```
# Try to attack the safe server
python3 scripts/06_verify_safe.py
# Result: All attacks BLOCKED!
```

**Lesson:** Compiler protections are great, but secure coding (avoiding `strcpy`) is the ultimate fix.

## PART 6: Bonus — Buffer Overflow Across Software Types

---

Buffer overflow is not just a C tutorial problem — it affects any software that uses native (C/C++) code.

### 6.1 Web Applications

**How BOF affects web:** The web server itself (Apache, Nginx), or libraries it uses (OpenSSL), can have buffer overflows.

**Famous Example:** \*Heartbleed (CVE-2014-0160)\* was a buffer over-read in OpenSSL. A malicious HTTP request allowed attackers to read 64KB of server memory, exposing passwords and private keys.

### 6.2 Mobile Applications

**How BOF affects mobile:** Android apps are written in memory-safe Java/Kotlin, but for performance, they rely on the **Java Native Interface (JNI)** to call C/C++ code. Vulnerabilities in these native libraries lead to remote code execution on the device.

### 6.3 Desktop Applications

**How BOF affects desktop:** Programs like Adobe Reader, VLC, and browsers parse complex file formats. A malformed PDF or video file can trigger an overflow during parsing, giving an attacker full control simply because the user opened a document.

### 6.4 IoT / Embedded Systems

**How BOF affects IoT:** Routers, IP cameras, and smart devices run tiny operating systems on weak processors. Their firmware is written in C and is almost always compiled **without** modern protections (no ASLR, no DEP, no canaries).

**Famous Example:** The \*Mirai Botnet\* compromised millions of IoT devices via simple buffer overflows and default credentials to launch massive DDoS attacks.

---

## References

- [Smashing The Stack For Fun And Profit \(Aleph One\)](#)
  - [ROP Emporium](#)
  - [Pwntools Documentation](#)
  - [CWE-119: Improper Restriction of Operations within the Bounds of a Memory Buffer](#)
- 

**Repository:** [https://github.com/samir22418/buffer\\_overflow](https://github.com/samir22418/buffer_overflow)